Technical note

Key decisions

Six agents share a SQLite blackboard of typed records instead of passing prompts along:

- the Orchestrator writes tasks;
- the Observability and Dependency specialists, running in parallel, write facts that must cite evidence IDs the tools really returned;
- Reasoning writes 2–4 ranked hypotheses;
- the Critic writes a verdict.

Separating hypothesis generation from falsification is the point of the design. The Critic either approves the top hypothesis, or blocks it once, naming the competing explanation and the specialist who can settle it. Python turns that into a follow-up task.

On top of the blackboard, agents can also message each other through a capped mailbox (six new messages per run, replies free). Python still decides who speaks next.

Routing, budgets, retries and grounding checks are plain Python; the model never decides when to stop. Tools live behind a custom MCP server with deterministic failure injection. A gateway validates, retries, deduplicates and budgets every call, for both systems alike.

Incidents come from a seeded generator, with the answer key in a schema the agents cannot read.

Trade-offs (12 story incidents, eval-v4)

	Team	Single agent
Top guess right	11/12	8/12
Decisive evidence found	100%	83%
Tokens per run	522K	89K
Time per run	422 s	69 s
Brier score	0.32	0.25

“Top guess right” includes reports that ended inconclusive. The team’s top guess was better, but on four identifiable incidents it hit the round limit without Critic approval and reported inconclusive at low confidence. Counting committed answers, the systems are level, and the team costs six times the tokens.

Against eval-v3 (blackboard only, no company background), the team’s top guess rose from 9 to 11, but its inconclusive reports rose from 2 to 5 and tokens from 322K to 522K. The single agent fell from 11 to 8 with only prompt wording and background changed, so much of this movement is run-to-run noise.

Limitations

- Twelve incidents from one model (Haiku), one run each. Differences of one or two incidents are noise.
- Synthetic stories move only the metrics they name, so some signals contradict. One incident was corrected after its first run (disclosed).
- `claude -p` runs as a subprocess, which adds process overhead, and has no temperature control. Replies are cached for replay.
- The caps (3 Critic reviews, 40 tool calls) limit long multi-hop chains. The mailbox refused about ten messages per run.
- The messaging pattern was tried on one of these incidents first.

Improvements

- Let the Critic close its own objection when new facts answer it; skip the Critic loop when the first ranking is decisive.
- Score service and category separately.
- Generate correlated metrics per failure mode.
- Add a human approval gate before any remediation is executed.

Incomplete functionality and external dependencies

Incomplete functionality

- **Simulated incidents only.** The system reads generated data and is not connected to real production systems. Live monitoring was not built.
- **Fixes are not applied.** The report recommends actions, but nothing carries them out, and there is no human approval step.
- **Past-incident search is by keyword,** not by meaning.
- **Partial evaluation.** 37 incidents are generated. The final evaluation covers 12 of them, with one run per system and one model (Haiku).
- **The viewer has no login and no daily limit on reruns.**
- **Setup was tested on Windows only.** The Linux and macOS setup (`make setup`) has the same steps but was not run.

External dependencies

Dependency	Why it is needed
Docker (or PostgreSQL 16)	Holds the incident data, the answer key and the evaluation logs.
Claude CLI, logged in	Every agent step is a `claude -p` call, so the plan's usage limits cap how many runs fit in a window.
Python 3.11+	Runs the system, with the packages pinned in `pyproject.toml` (MCP, LangGraph, Pydantic, psycopg, FastAPI).
Internet access	Needed for Claude calls. The viewer also loads its charts, diagrams and fonts from public sites.
Tailscale (optional)	Only used to share the viewer through a link.

What would make it perform better

- **Add RAG.** Search past incidents and runbooks by meaning, so agents start from similar cases. This adds overhead: an index to build and keep current, and more tokens per step.
- **Call the API directly instead of claude -p.** This removes the subprocess per step and the plan's usage limits, and gives direct control over model settings and prompt caching.
- **Supervised prompt-level tuning.** Use the labelled incidents to compare prompt versions on a training set, then confirm the best one on incidents held back for testing. The answer key stays out of the agents' reach at run time.